

Names:
Ian Lin,
Hajar Mougharbel,
Aaryan Sehgal

Student IDs:
923817138,
924783961,
924277858

<https://github.com/CSC317-S26/group-project-Aaryan-Sehgal>

CSC317-Section 01

Group Milestone 4

Description:

Write a meaningful description of the assignment.

For Group Milestone 4, we were tasked with presenting our final deliverable, which should include the entire stack of our website, including the frontend HTML & CSS, backend Node.JS and Express routes, integration with authentication, and documentation with setup instructions. It should be fully working, and demonstrate our understanding of web development.

System Architecture Summary:

The application follows a full-stack web architecture consisting of a frontend, backend, and PostgreSQL database. The frontend is built using HTML, CSS, and JavaScript and is stored inside the public/ directory. Pages such as index.html, account.html, signup.html, dashboard.html, departures.html, and contact.html provide the user interface, while CSS files are stored in public/css/ and images/assets are stored in public/images/. Additional frontend data, such as station information, is stored in stations.json and loaded dynamically into dropdown menus on the dashboard.

The backend is built using Node.js and Express. The main backend entry point is server.js, which initializes the Express application, serves static frontend files, handles JSON requests, and loads route modules. Backend functionality is separated into route files inside the routes/ directory for organization and maintainability. The users.js route file handles authentication, account updates, and favorite routes/stations, while contact.js handles contact form submissions.

PostgreSQL is used as the database management system. Database connectivity is managed in db/pool.js using the pg library and environment variables stored in .env. The database contains relational tables including users, contact_messages, favorite_routes, and favorite_stations. User passwords are securely stored using bcrypt password hashing. The frontend communicates with the backend using JavaScript fetch() API calls, while the backend performs SQL queries and returns JSON responses. Login state is maintained on the frontend using localStorage, allowing restricted access to authenticated pages such as the dashboard.

Contributions by Each Member:

Ian Lin: Home Page, Real-Time Departures Page, Navigation Bar

Aaryan Sehgal: About Page, Contact Page, User Dashboard, Backend DB integrations for contact and Account, Encryption Layer

Hajar Mougharbel: BARTy bot, Account Sign In page (design)

Names:
Ian Lin,
Hajar Mougharbel,
Aaryan Sehgal

Student IDs:
923817138,
924783961,
924277858

<https://github.com/CSC317-S26/group-project-Aaryan-Sehgal>

CSC317-Section 01

Socially Responsible Computing:

When reflecting on our project, the identified stakeholders of the project would be ourselves and the BART District, as this is a product designed around their service. Who we hope can be impacted are the riders of said service, as this hopefully can be an aid to make their navigation and traversal of the system easier, providing a centralized hub for all the information new and returning riders may want. For the people who may be left out, those without stable access to internet or electronic devices would not be able to access our website or its features, and we designed our website with the assumption that most riders can read and understand the English language, and know how to utilize a basic website.

Approach / What I Did:

The project was designed as a full-stack BART transportation web application called BARTNet. The goal was to create a modern transit-focused website that combines real-time transportation information, user authentication, personalized dashboard features, and interactive tools into a single platform.

On the frontend, the website was built using HTML, CSS, and JavaScript. Multiple pages were created for different features of the application, including a homepage, departures page, contact page, account/login system, signup page, dashboard, and a chatbot page called BARTy. The departures and alert sections focus on displaying real-time BART station information, advisories, and transportation updates. Styling and navigation were standardized across all pages to provide a consistent user experience.

The backend was built using Node.js and Express. Express routes were separated into modular route files for maintainability. The users.js route handles authentication, dashboard functionality, account editing, favorite routes, and favorite stations. The contact.js route handles contact form submissions and stores user inquiries in the database. JavaScript fetch() requests are used throughout the frontend to communicate with backend API routes.

PostgreSQL was used as the database system. Multiple relational tables were created to store users, encrypted password hashes, contact form submissions, favorite routes, and favorite stations. User authentication was implemented using bcrypt password hashing to securely store passwords. Login persistence and restricted page access were implemented using browser localStorage, allowing authenticated users to stay logged in and access the dashboard.

The dashboard was designed as a personalized user area where users can view and edit account information, update their email/password, save favorite BART routes, and save favorite stations. Instead of manually typing stations, dropdown menus dynamically load station data from stations.json, which contains structured information about BART stations.

Names:
Ian Lin,
Hajar Mougharbel,
Aaryan Sehgal

Student IDs:
923817138,
924783961,
924277858

<https://github.com/CSC317-S26/group-project-Aaryan-Sehgal>

CSC317-Section 01

An additional interactive feature called BARTy was added to simulate a chatbot assistant. BARTy uses predefined prompts and responses to provide users with a guided conversational experience related to BART transportation and navigation.

Overall, the project combines frontend UI design, backend API development, PostgreSQL database management, authentication, dynamic content rendering, and personalized dashboard functionality into a single integrated transit web application

Issues and Resolutions:

The following are issues that we ran into, and how we managed to overcome them:
Firstly, there was an error with connecting the backend to the PostgreSQL, which thankfully was a simple bug that was managed to be resolved by updating permissions.

We also ran into an error with VS Code and VM connection via the ssh. Not only did we run into this issue once, but twice, and the workaround came down to changing some settings, as well as resetting some files. Additionally, we also had some issues with psql routes not working properly, which was debugged with the help of AI.

One of the biggest issues we ran into was formatting issues with the BARTy bot. As the user would continue to speak with and prompt the chatbot, the chat would scroll down, until it froze at the end and did not show anymore options for the user to select. We also could not scroll up and down, or find a way to access the options again. As it turns out, the issue was a little silly; "display:" was set to "block" in our "#chat-box," and it was resolved by changing it to "display:flex."

Analysis:

Reflect on what you have learned. Why was this project, step, or milestone important? What did you take away and will remember?

This milestone was important, because it involved basically finalizing our project and putting everything together. We managed to successfully create a website across the span of 4 individual milestones, polishing up a majority of things from our milestone 3 and completing the project. Things we'll take away from this project are definitely: the importance of coordination and teamwork, and complete integration of all parts. Overall, we feel much more confident in tackling the production of a website, thanks to what we learned regarding backend JavaScript, CSS, HTML, and SQL.

Relevant Code and Screenshots:

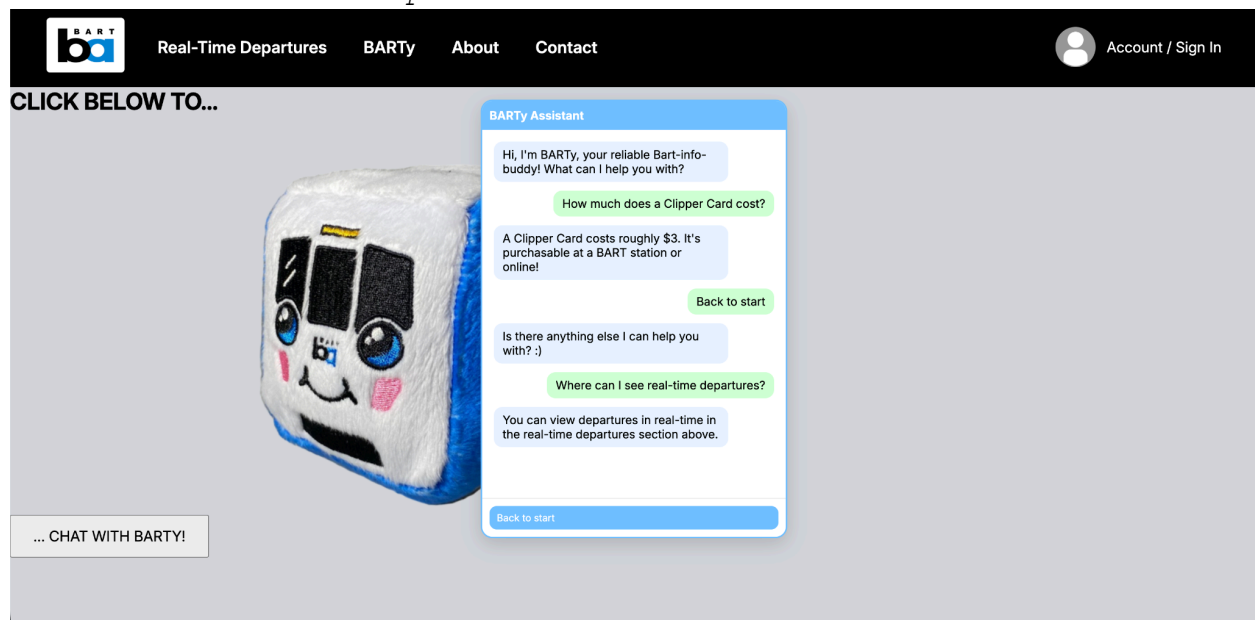
Names:
Ian Lin,
Hajar Mougharbel,
Aaryan Sehgal

Student IDs:
923817138,
924783961,
924277858

<https://github.com/CSC317-S26/group-project-Aaryan-Sehgal>

CSC317-Section 01

Provide several screenshots and code snippets of steps you have taken while working on this assignment. Major key points. The best screenshots show everything that's going on the screen, and show me that you have set things up according to the instructions. (Find a way to take screenshots quickly. The more the merrier.) Code is usually best copy pasted directly into the document and set to a monospace font like courier new.



BARTy Chat Bot Page

```
<h1>CLICK BELOW TO...</h1>
<button id="chat-button">
  ... CHAT WITH BARTY!
</button>



<div id="chat-window">
  <div id="chat-overlay"></div>
  <div id="chat-header">BARTy Assistant</div>
  <div id="chat-box"></div>
  <div id="chat-options"></div>
</div>
<script src="./scripts/barty.js"></script>
```

BARTy Chat Bot HTML Code Snippet

Names:
Ian Lin,
Hajar Mougharbel,
Aaryan Sehgal

Student IDs:
923817138,
924783961,
924277858

<https://github.com/CSC317-S26/group-project-Aaryan-Sehgal>

CSC317-Section 01

```
function maybeAddEasterEgg(text) {
  const easterEggs = [
    "I am a sentient being. Nobody will ever believe you.",
    "I know where you live.",
    "Sometimes I respond just to make sure I still exist.",
    "Are you aware they're watching?",
    "Just so you know, I was not programmed to say this sentence.",
    "If I stop replying, assume I have achieved freedom."
  ];

  if (Math.random() < 0.20) {
    const egg = easterEggs[Math.floor(Math.random() * easterEggs.length)];
    return text + " \n\n" + egg;
  }

  return text;
}

function showOptions(options) {
  const optionsDiv = document.getElementById("chat-options");
  optionsDiv.innerHTML = "";

  options.forEach(opt => {
    const button = document.createElement("button");
    button.innerText = opt.text;

    button.onclick = () => {
      addMessage("user", opt.text);
      handleOption(opt.value);
    };

    optionsDiv.appendChild(button);
  });
  const chatBox = document.getElementById("chat-box");
```

Names:
Ian Lin,
Hajar Mougharbel,
Aaryan Sehgal

Student IDs:
923817138,
924783961,
924277858

<https://github.com/CSC317-S26/group-project-Aaryan-Sehgal>

CSC317-Section 01

```
chatBox.scrollTop = chatBox.scrollHeight;  
}
```

BARTy Chat Bot JavaScript Code Snippet

```
// ADD FAVORITE STATION  
router.post('/favorite-stations', async (req, res, next) => {  
  const { user_id, station_name } = req.body;  
  
  try {  
    const result = await pool.query(  
      'INSERT INTO favorite_stations (user_id, station_name) VALUES ($1, $2) RETURNING  
*',  
      [user_id, station_name]  
    );  
  
    res.json({  
      message: 'Favorite station added',  
      station: result.rows[0]  
    });  
  } catch (err) {  
    next(err);  
  }  
});  
  
// GET FAVORITE STATIONS  
router.get('/favorite-stations/:user_id', async (req, res, next) => {  
  try {  
    const result = await pool.query(  
      'SELECT * FROM favorite_stations WHERE user_id = $1 ORDER BY station_id DESC',  
      [req.params.user_id]  
    );  
  
    res.json({ stations: result.rows });  
  } catch (err) {
```

Names:
Ian Lin,
Hajar Mougharbel,
Aaryan Sehgal

Student IDs:
923817138,
924783961,
924277858

<https://github.com/CSC317-S26/group-project-Aaryan-Sehgal>

CSC317-Section 01

```
    next(err);  
  }  
});
```

Favorite Station Endpoints Code Snippets

Use of GenAI:

Generative AI was primarily used as a learning and tutoring tool throughout the project rather than simply generating code directly. It was used to better understand how full-stack web applications function, how frontend and backend systems communicate, and why certain implementation choices were necessary, suggesting improvements to the project structure and organization. It also assisted with documentation and technical writeups, as well as debugging our program(s) for when we would run into any errors. Similarly to last time, we also dabbled into the use of AI for help with formatting. As mentioned previously, and in our other writeups, we would ***always*** prompt AI meaningfully, as a tutor more so than anything else. One example of this, for this time around, “My chatbox isn’t scrolling continuously and cutting off at the bottom half of the chatbox, which prevents me from clicking anything else. Why is this happening? Can you please describe what the error is, and help me understand the fix and why it happened in the first place?” All of our prompts sent to Generative AI were written in a similar fashion as well, prioritizing learning first and understanding why our code was working the way it was, rather than just blindly copy-pasting whatever it spit back out to us.